

Functions

```
graph TD; Functions[Functions] --> Predefined[Predefined]; Functions --> UserDefined[User defined];
```

Predefined

- Library functions
- Declaration in header files
- Body in dll files

User defined

User customized functions to reduce complexity of big programs

C FUNCTIONS

By:
Rishi Kumar



WHY FUNCTIONS?

- Functions break large computing tasks into smaller ones
- Appropriate functions hide details of operation from parts of the program
- Enable programmers to build on what others have done instead of starting over from scratch

C FUNCTIONS

Definition: A function is a named, independent section of C code, that performs a specific task and optionally returns a value to the calling program or/and receives values(s) from the calling program.

- Basically there are two types of function:
 - **Predefined functions:** Functions such as puts() , gets() , printf() , scanf() etc are standard library functions. These functions are available in C standard library such as stdio.h, math.h, string.h etc.
 - **User-defined functions:** These function are created by user as per their own requirement.

Call a Function

// Create a function

```
void myFunction() {  
    printf("I just got executed!");  
}  
  
int main() {  
    myFunction(); // call the function  
    return 0;  
}
```

```
void myFunction() {  
    printf("I just got  
executed!");  
}
```

```
int main() {  
    myFunction();  
    myFunction();  
    myFunction();  
    return 0;  
}
```

PARAMETERS AND ARGUMENTS

```
returnType functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

```
void myFunction(char name[]) {  
    printf("Hello %s\n", name);  
}
```

```
int main() {  
    myFunction("ABC");  
    myFunction("XYZ");  
    myFunction("SUV");  
    return 0;  
}
```

```
void myFunction(char name[], int age)  
{  
    printf("Hello %s. You are %d  
years old.\n", name, age);  
}
```

```
int main() {  
    myFunction("ABC", 3);  
    myFunction("XYZ", 14);  
    myFunction("SUV", 30);  
    return 0;  
}
```

Pass Arrays as Function Parameters

```
void myFunction(int myNumbers[5]) {  
    for (int i = 0; i < 5; i++) {  
        printf("%d\n", myNumbers[i]);  
    }  
}
```

```
int main() {  
    int myNumbers[5] = {10, 20, 30, 40, 50};  
    myFunction(myNumbers);  
    return 0;  
}
```

RETURN VALUES

```
int myFunction(int x) {  
    return 5 + x;  
}  
  
int main() {  
    printf("Result is: %d", myFunction(3));  
    return 0;  
}
```

```
2. int myFunction(int x, int y) {  
    return x + y;  
}  
  
int main() {  
    int result = myFunction(5, 3);  
    printf("Result is = %d", result);  
    return 0;  
}
```

C FUNCTIONS

/*Example C program to find cube of any number using function */

```
#include <stdio.h>
```

```
/* Function declaration */
```

```
double cube(double num);
```

```
int main() {
```

```
int num;
```

```
double c;
```

```
/* Input number to find cube from  
user */
```

```
printf("Enter any number: ");
```

```
scanf("%d", &num);
```

```
c = cube(num);
```

```
printf("Cube of %d is %.2f", num,  
c);
```

```
return 0;
```

```
}
```

```
/* Function to find cube of any  
number */
```

```
double cube(double num)
```

```
{
```

```
return (num * num * num);
```

```
}
```



```

#include<stdio.h>

long cube(long x);    //function prototype

long input, answer;

int main( )
{ printf("Enter values: ");
  scanf("%d", &input);
  answer= cube(input);    //function call
  printf("The cub of %ld is %ld\n", input, answer);
  return 0;    }

long cube(long x)    //function definition
{ long x_cubed;
  x_cubed = x*x*x;
  return x_cubed;  }

```

❖Function Declaration.

❖Function Call

❖Function Definition.

C FUNCTIONS

- The following statement call `cube ()` function, bringing along the value assigned to the `num` variable.

```
c = cube (num) ;
```

- When this statement is executed, program jump to the `cube ()` function definition.
- After the execution completed, the `cube ()` function returns to the caller program (`main ()`), assigning the returned value
- In this program, the `scanf ()` and `printf ()` are examples of the standard predefined functions.

C FUNCTIONS

- Basically a function has the following characteristics:
 1. Named with unique name .
 2. Performs a specific task - Task is a discrete job that the program must perform as part of its overall operation, such as sending a line of text to the printer, sorting an array into numerical order, or calculating a cube root, etc.
 3. Independent - A function can perform its task without interference from or interfering with other parts of the program.
 4. May receive values from the calling program (caller) - Calling program can pass values to function for processing whether directly or indirectly (by reference).
 5. May return a value to the calling program – the called function may pass something back to the calling program.

C FUNCTIONS

Function Mechanism

- C program does not execute the statements in a function until the function is called.
- When it is called, the program can send information to the function in the form of one or more arguments although it is not a mandatory.
- Argument is a program data needed by the function to perform its task.
- When the function finished processing, program returns to the same location which called the function.

C FUNCTIONS

Function Definition

Is the actual function body, which contains the code that will be executed as shown below (previous example).

/* Function to find cube of any number */

```
double cube(double num)  
{  
return (num * num * num);  
}
```

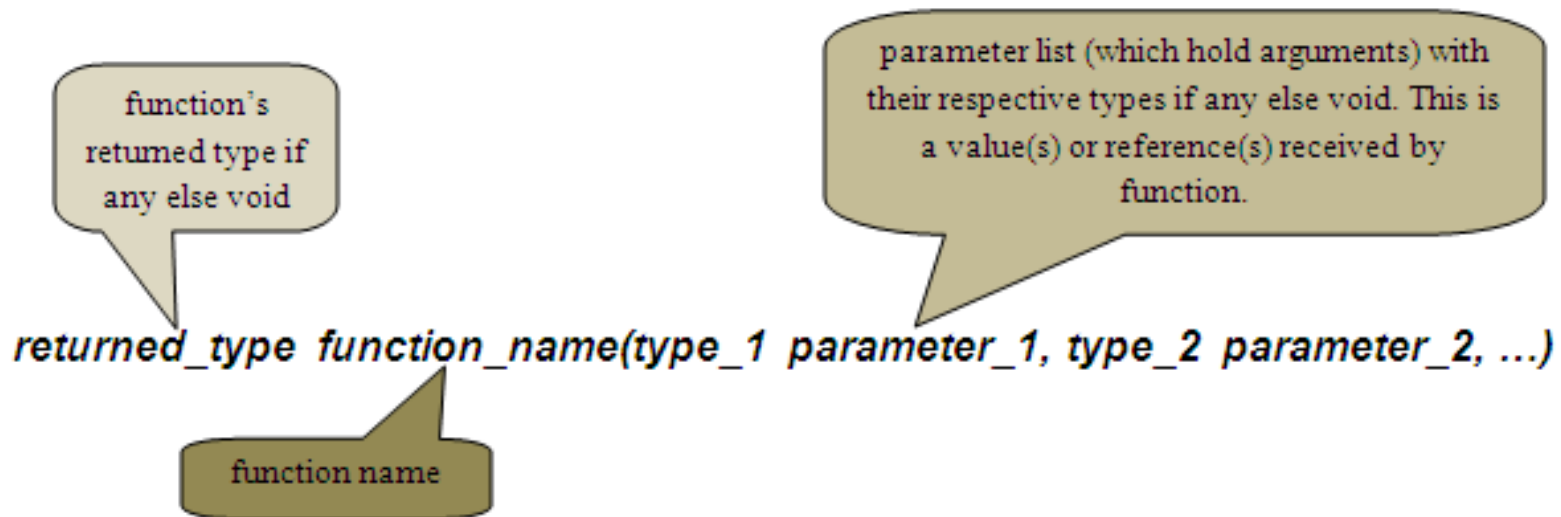
C FUNCTIONS

- First line of a function definition is called the function header, should be identical to the function prototype, except the semicolon.
- Although the argument variable names (`num` in this case) were optional in the prototype, they must be included in the function header.
- Function body, containing the statements, which the function will perform, should begin with an opening brace and end with a closing brace.
- If the function returns data type is anything other than `void` (nothing to be returned), a `return` statement should be included, returning a value matching the return data type (`int` in this case).

C FUNCTIONS

The Function header

The first line of every function definition is called function header. It has 3 components, as shown below.



1. **Function return type** - Specifies the data type that the function should return to the caller program. Can be any of C data types: `char`, `float`, `int`, `long`, `double`, `pointers` etc. If there is no return value, specify a return type of `void`. For example,

```
int    calculate_yield(...) // returns an int type
float  mark(...)           // returns a float type
void   calculate_interest(...) // returns nothing
```

C FUNCTIONS

- Function name - Can have any name as long as the rules for C variable names are followed and must be unique.
- Parameter list - Many functions use arguments, the value passed to the function when it is called. A function needs to know the data type of each argument. Argument type is provided in the function header by the parameter list. Parameter list acts as a placeholder.

C FUNCTIONS

- For each argument that is passed to the function, the parameter list must contain one entry, which specifies the type and the name.
- For example,

```
void myfunction(int x, float y, char z)
void yourfunction(float myfloat, char mychar)
int  ourfunction(long size)
```

- The first line specifies a function with three arguments: type `int` named `x`, type `float` named `y` and type `char` named `z`.
- Some functions take no arguments, so the parameter list should be `void` or empty such as,

```
long thefunction(void)
void testfunct(void)
int  zerofunct()
```

C FUNCTIONS

- Parameter is an entry in a function header. It serves as a placeholder for an argument. It is fixed, that is, do not change during execution.
- The argument is an actual value passed to the function by the caller program. Each time a function is called, it can be passed with different arguments.
- A function must be passed with the same number and type of arguments each time it is called, but the argument values can be different.

C FUNCTIONS

The Function Body

- Enclosed in curly braces, immediately follows the function header.
- Real work in the program is done here.
- When a function is called execution begins at the start of the function body and terminates (returns to the calling program) when a `return` statement is encountered or when execution reaches the closing braces `{ }`.
- Variable declaration can be made within the body of a function.
- Which are called local variables. The scope, that is the visibility and validity of the variables are local.
- Local variables are the variables apply only to that particular function, are distinct from other variables of the same name (if any) declared elsewhere in the program outside the function.
- It is declared, initialized and use like any other variable.
- Outside of any functions, those variables are called global variables.

C FUNCTIONS

```
#include ...
```

```
/* function prototype */
```

```
int funct1(int);
```

```
int main()
```

```
{
```

```
    /* function call */
```

```
    int y = funct1(3);
```

```
    ...
```

```
}
```

```
/* Function definition */
```

```
int funct1(int x)
```

```
{...}
```

But it is OK if we directly declare and define the function before `main()` as shown below.

```
#include ...
```

```
/* declare and define */
```

```
int funct1(int x)
```

```
{
```

```
    ...
```

```
}
```

```
int main()
```

```
{
```

```
    /* function call */
```

```
    int y = funct1(3);
```

```
    ...
```

```
}
```

Three rules govern the use of variables in functions:

1. To use a variable in a function, we must declare it in the function header or the function body.
2. For a function to obtain a value from the calling program (caller), the value must be passed as an argument (the actual value).
3. For a calling program (caller) to obtain a value from function, the value must be explicitly returned from the called function (callee).

C FUNCTIONS

The Function Statements

- Any statements can be included within a function, however a function may not contain the definition of another function.
- For examples: if statements, loop, assignments etc are valid statements.

Returning a Value

- A function may or may not return a value.
- If function does not return a value, then the function return type is said to be of type void.
- To return a value from a function, use return keyword, followed by C expression.
- The value is passed back to the caller.
- The return value must match the return data type.
- A function can contain multiple return statements.

C FUNCTIONS

- It is recommended to include a function prototype in your C program to avoid ambiguity.
- Identical to the function header, with semicolon (;) added at the end.
- Function prototype includes information about the function's return type, name and parameters' list and type.
- The general form of the function prototype is shown below,

function_return_type function_name(type parameter1, type parameter2,..., type parameterN)

- An example of function prototype,

double cube(double);

C FUNCTIONS

- Function prototype provides the C compiler the name and arguments of the functions and must appear before the function is used or defined.
- It is a model for a function that will appear later, somewhere in the program.
- From the previous prototype example, 'we' know the function is named `cube`, it requires a variable of the type `double`, and it will return a value of type `double`.
- Then, the compiler can check every time the source code calls the function, verify that the correct number and type of arguments are being passed to the function and check that the return value is returned correctly.
- If mismatch occurs, the compiler generates an error message enabling programmers to trap errors.
- A function prototype need not exactly match the function header.
- The optional parameter names can be different, as long as they are the same data type, number and in the same order.
- But, having the name identical for prototype and the function header makes source code easier to understand.

C FUNCTIONS

- Normally placed before the start of `main()` but must be before the function definition.
- Provides the compiler with the description of a function that will be defined at a later point in the program.
- Includes a return type which indicates the type of variable that the function will return.
- And function name, which normally describes what the function does.
- Also contains the variable types of the arguments that will be passed to the function.
- Optionally, it can contain the names of the variables that will be returned by the function.
- A prototype should always end with a semicolon (`;`).

C FUNCTIONS

■ **Passing Arguments to a Function**

- In order function to interact with another functions or codes, the function passes arguments.
- The called function receives the values passed to it and stores them in its parameters.
- List them in parentheses following the function name.

C FUNCTIONS

- The number of arguments and the type of each argument must match the parameters in the function header and prototype.
- If the function takes multiple arguments, the arguments listed in the function call are assigned to the function parameters in order.
- The first argument to the first parameter, the second argument to the second parameter and so on as illustrated below.

The diagram illustrates the mapping of arguments from a function call to its prototype. It shows two lines of code: `function1(a, b, c);` on the top line and `void function1(int x, int y, int z)` on the bottom line. Three arrows originate from the arguments 'a', 'b', and 'c' in the function call and point to the parameters 'x', 'y', and 'z' in the function prototype, respectively, demonstrating the order of argument passing.

- Basically, there are two ways how we can pass something to function parameters,
 1. Passing by value.
 2. Passing by reference using array and pointer.

C FUNCTIONS

- The following table summarizes the various ways of using functions.
- We can categorize functions by whether or not arguments are passed to them.
- Or we can categorize them by whether or not values are received from them.
- Intersecting these two methods of categorizing functions, we come up with four basic methods of writing and using functions.

C FUNCTIONS

	Do not pass argument	Do pass arguments
No return	<pre>void main(void) { TestFunc(); ... } void TestFunc(void) { // receive nothing // and nothing to be // returned }</pre>	<pre>void main(void) { TestFunc(123); ... } void TestFunc(int i) { // receive something and // the received/passed // value just // used here. Nothing // to be returned. }</pre>
With a return	<pre>void main(void) { x = TestFunc(); ... } int TestFunc(void) { // received/passed // nothing but need to // return something return 123; }</pre>	<pre>void main(void) { x = TestFunc(123); ... } int TestFunc(int x) { // received/passed something // and need to return something return (x + x); }</pre>

C FUNCTIONS

Example 1: No arguments passed and no return value

```
#include <stdio.h>

void checkPrimeNumber();

int main() {
    checkPrimeNumber(); // argument
                        // is not passed
    return 0;
}

// return type is void meaning
// doesn't return any value

void checkPrimeNumber() {
    int n, i, flag = 0;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
```

```
    for(i=2; i <= n/2; ++i) {
        if(n%i == 0)
        {
            flag = 1;
        }
    }

    if (flag == 1)
        printf("%d is not a prime
        number.", n);

    else printf("%d is a prime
    number.", n);
}
```

C FUNCTIONS

Example 2: No arguments passed but a return value

```
#include <stdio.h>

int getInteger();

int main() {
    int n, i, flag = 0;
    // no argument is passed
    n = getInteger();
    for(i=2; i<=n/2; ++i) {
        if(n%i==0)
        {
            flag = 1;
            break;
        }
    }
}
```

```
    if (flag == 1)
        printf("%d is not a prime number.", n);
    else
        printf("%d is a prime number.", n);
    return 0; }

// returns integer entered by the user
int getInteger()
{
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    return n;
}
```

C FUNCTIONS

Example 3: Argument passed but no return value

```
#include <stdio.h>

void checkPrimeAndDisplay(int n);

int main() {
    int n;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    // n is passed to the function
    checkPrimeAndDisplay(n);

    return 0;
}

// return type is void meaning doesn't
// return any value

void checkPrimeAndDisplay(int n)
{
    int i, flag = 0;
```

```
    for(i=2; i <= n/2; ++i)
    {
        if(n%i == 0)
        {
            flag = 1;
            break;
        }
    }

    if(flag == 1)
        printf("%d is not a prime number.", n);
    else
        printf("%d is a prime number.", n);
}
```

C FUNCTIONS

Example 4: Argument passed and a return value

```
#include <stdio.h>

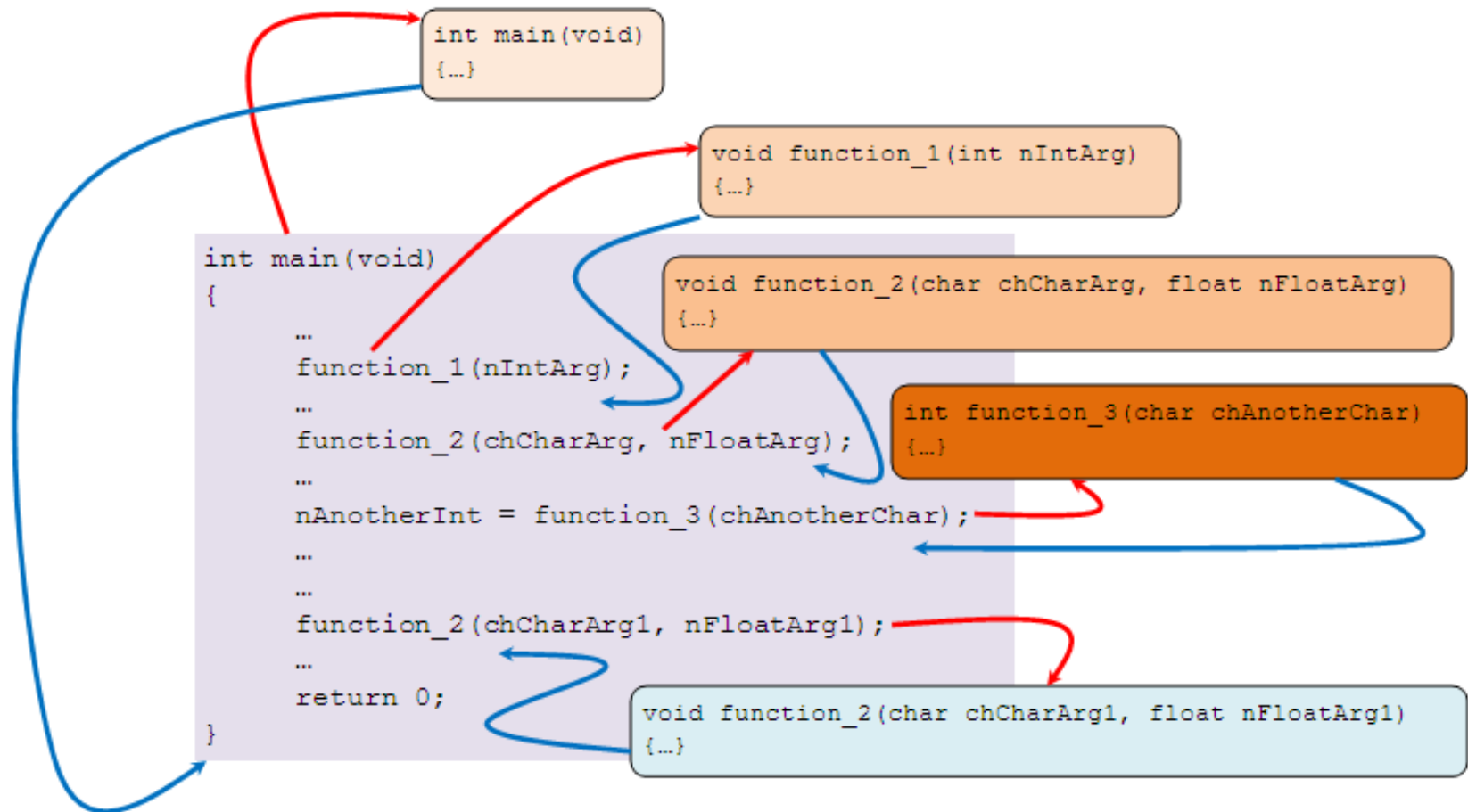
int checkPrimeNumber(int n);
int main() {
    int n, flag;

    printf("Enter a positive integer: ");
    scanf("%d",&n);

    // n is passed to the
    checkPrimeNumber() function
    // the returned value is assigned to the
    flag variable
    flag = checkPrimeNumber(n);

    if(flag == 1)
        printf("%d is not a prime number",n);
    else printf("%d is a prime number",n);
    return 0; }

// int is returned from the function int
checkPrimeNumber(int n) {
    int i;
    for(i=2; i <= n/2; ++i)
    {
        if(n%i == 0)
            return 1;
    }
    return 0;
}
```

C FUNCTIONS

Recursive Function

- We cannot define function within function
- But, we can call the same function within the function.
- A function that calls itself is known as a recursive function and this technique is known as recursion.

C FUNCTIONS

How does recursion work?

```
void recurse()
{
    ... ..
    recurse();
    ... ..
}

int main()
{
    ... ..
    recurse();
    ... ..
}
```

The diagram illustrates the flow of recursive calls. A line from the `recurse()` call inside the `main()` function points to the `recurse()` function definition. Another line from the `recurse()` call inside the `recurse()` function loops back to the line just above it, labeled "recursive call".

C FUNCTIONS

- The recursion continues until some condition is met to prevent it.
- To prevent infinite recursion, **if...else statement** (or similar approach) can be used where one branch makes the recursive call, and other doesn't.

C FUNCTIONS

Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>

int sum(int n);

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);

    printf("sum = %d", result);

    return 0;
}
```

```
int sum(int n) {
    if (n != 0)
        // sum() function calls itself
        return n + sum(n-1);
    else
        return n;
}
```

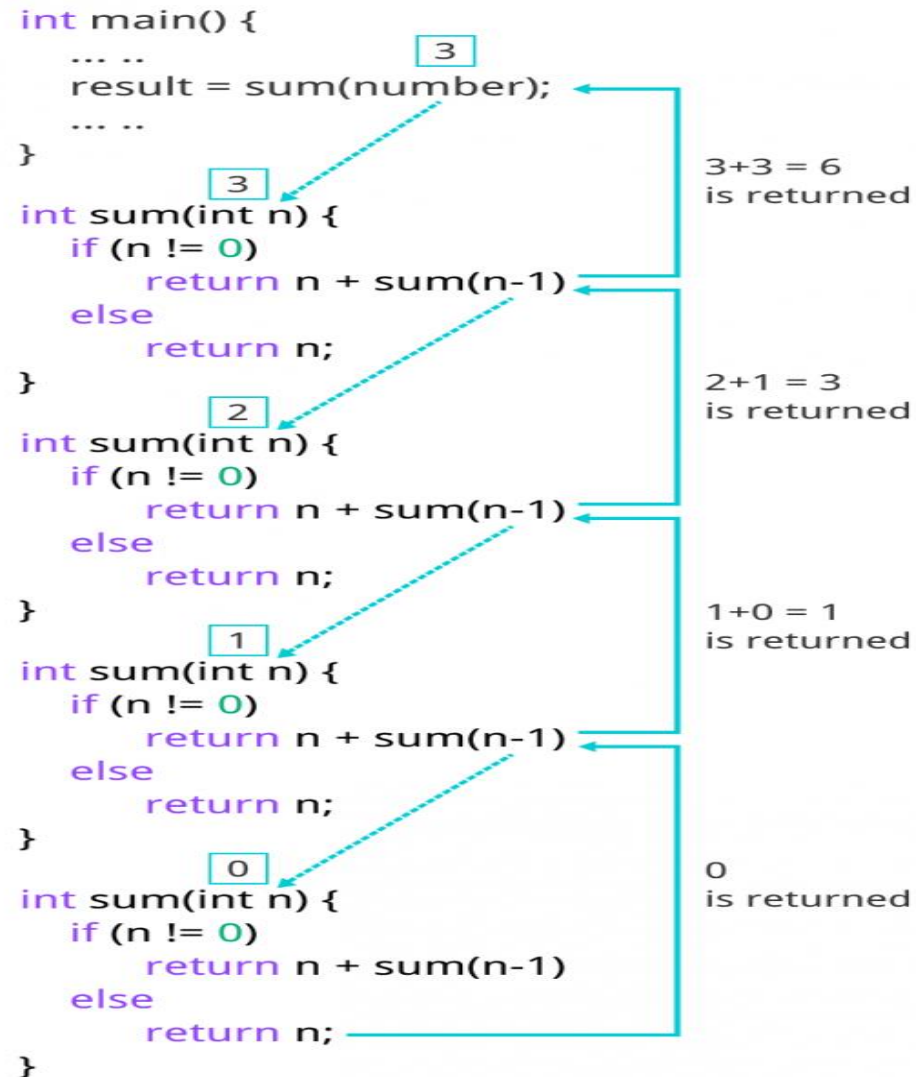
Output

```
Enter a positive integer:3
sum = 6
```

C FUNCTIONS

- Initially, the `sum()` is called from the `main()` function with number passed as an argument.
- Consider, the value of `n` inside `sum()` is 3 initially.
 - During the next function call, 2 is passed to the `sum()` function. This process continues until `n` is equal to 0.
- When `n` is equal to 0, the `if` condition fails and the `else` part is executed returning the sum of integers ultimately to the `main()` function.

C FUNCTIONS



C FUNCTIONS

- **Advantages of Recursion**

- Recursion makes program elegant

- **Disadvantages of Recursion**

- Use loops instead as, recursion is usually much slower

FACTORIAL OF A NUMBER USING RECURSION

```
#include<stdio.h>

long int multiplyNumbers(int n);

int main() {
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    printf("Factorial of %d = %ld", n, multiplyNumbers(n));
    return 0;
}

long int multiplyNumbers(int n) {
    if (n>=1)
        return n*multiplyNumbers(n-1);
    else
        return 1;
}
```

FUNCTION CALL

- The number of arguments and the type of each argument must match the parameters in the function header and prototype.
- If the function takes multiple arguments, the arguments listed in the function call are assigned to the function parameters in order.
- The first argument to the first parameter, the second argument to the second parameter and so on as illustrated below:

```
function1(a, b, c);
```

```
void function1(int x, int y, int z)
```

A diagram consisting of three arrows pointing from the arguments 'a', 'b', and 'c' in the function call 'function1(a, b, c);' to the parameters 'x', 'y', and 'z' in the function prototype 'void function1(int x, int y, int z)'. The first arrow points from 'a' to 'x', the second from 'b' to 'y', and the third from 'c' to 'z'.

- Basically, there are two ways how we can pass something to function parameters,
 - Passing by value.
 - Passing by reference using pointer.

CALL BY VALUE

- In call by value, value being passed to the function is locally stored by the function parameter in stack memory location.
- If you change the value of function parameter, it is changed for the current function only.
- It will not change the value of variable inside the caller method such as main().

```
#include <stdio.h>
#include <conio.h>
void change(int num) {
    printf("Before adding value inside function num=%d \n",num);
    num=num+100;
    printf("After adding value inside function num=%d \n", num);
}
int main() {
    int x=100;
    clrscr();
    printf("Before function call x=%d \n", x);
    change(x);//passing value in function
    printf("After function call x=%d \n", x);
    getch();
    return 0;
}
```

CALL BY REFERENCE

- In call by reference, original value is modified because we pass reference (address).

```
#include <stdio.h>
#include <conio.h>
void change(int *num)
{
    printf("Before adding value inside function num=%d \n", *num);
    (*num) += 100;
    printf("After adding value inside function num=%d \n", *num);
}
int main()
{
    int x=100;
    clrscr();
    printf("Before function call x=%d \n", x);
    change(&x);
    //passing reference in function
    printf("After function call x=%d \n", x);
    getch();
    return 0;
}
```

REFERENCES

- *Online Pdf, Let Us C – BPB Publications, Yashavant P. Kanetkar*
- *Online Pdf, The C Programming Language, 2nd Edition, Brian W. Kernighan, Dennis M. Ritchie, 1988*
- <https://www.programiz.com/cprogramming/c-recursion>
- https://www.w3schools.com/c/c_functions_parameters.php
- <https://www.geeksforgeeks.org/c-functions/>